Upcoming Events (RSVP) Page

Theme-Based Options / Finalization Page

Penn State Coding Club - Student Club Portal

IST 256 | Group 1

Github:

Group Members

Adam Nisenson

Elliot Buona

Zizhou Xiang

Nasser Almalki

Jason Lin

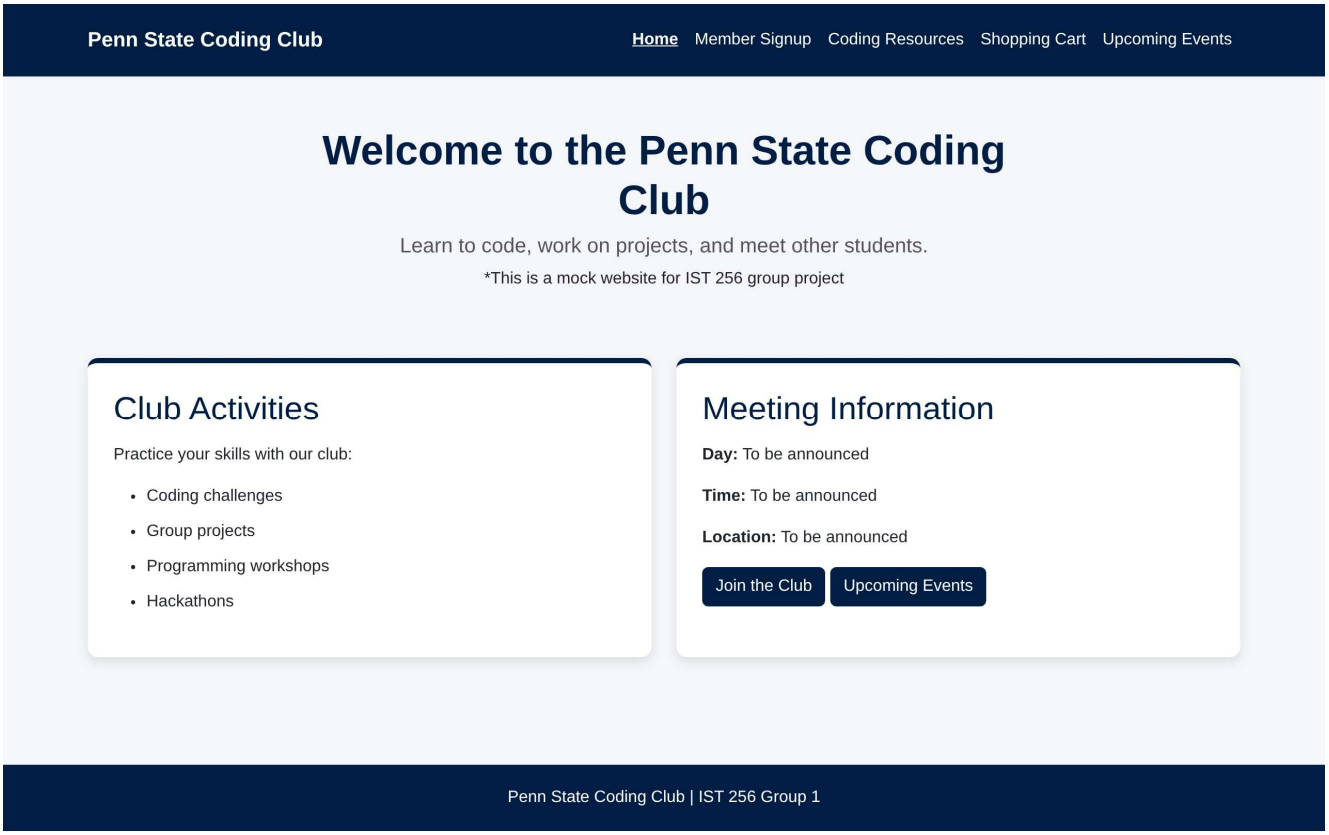
Madina Aliyeva

1 November 2026

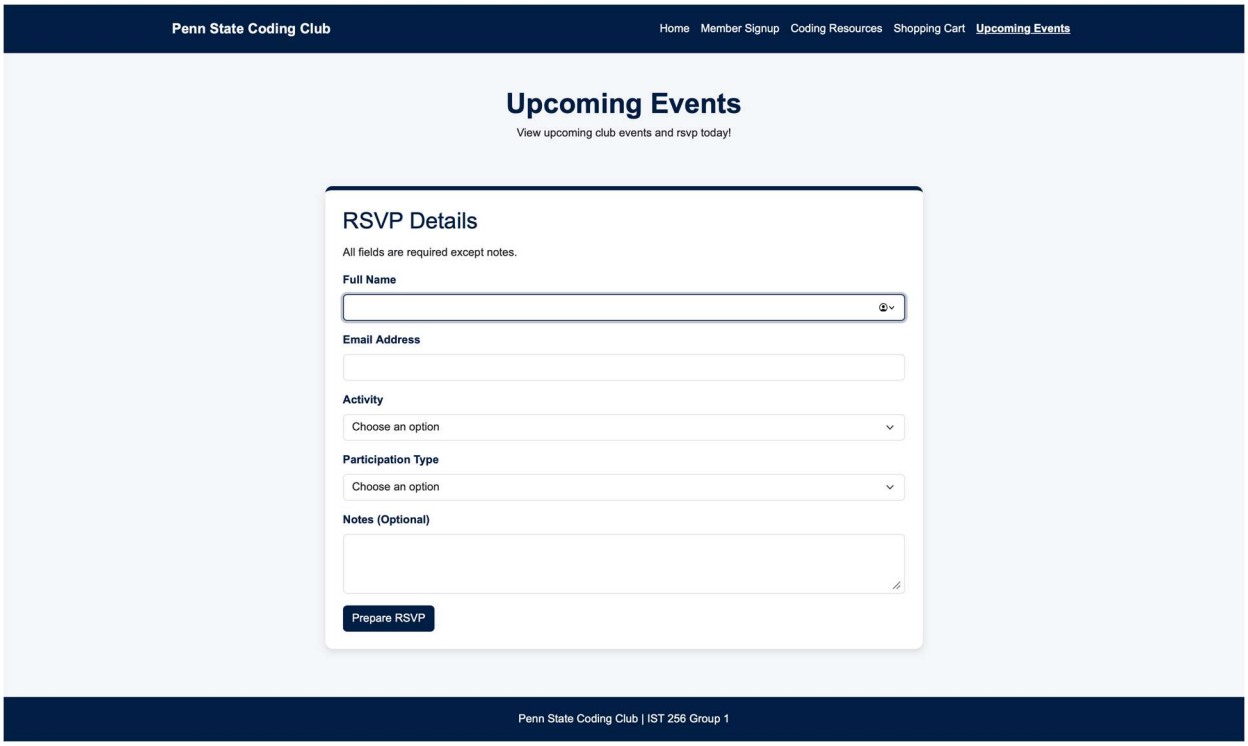
Group Contributions:

1. Upcoming Events (RSVP) Page

The Upcoming Events page is the finalization step for the club portal: members RSVP to a club workshop. Every page's bootstrap navbar links to Home, Member Signup, Coding Resources, Shopping Cart, and Upcoming Events, and the homepage's Meeting Information card also has an Upcoming Events button. The RSVP form is a React component inside a Bootstrap card styled by the shared CSS theme.



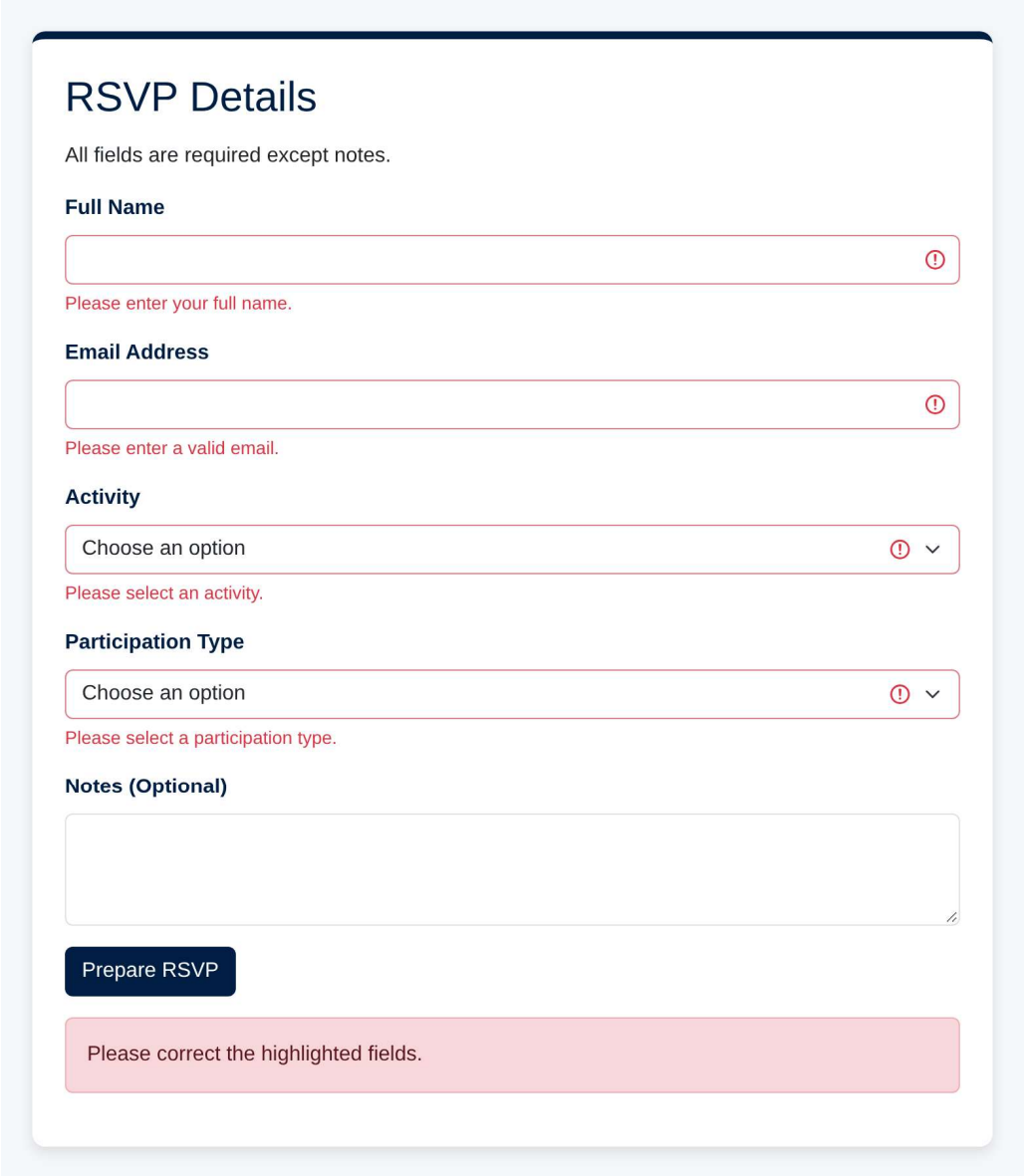
Homepage with Upcoming Events in the navigation menu and the Meeting Information card.



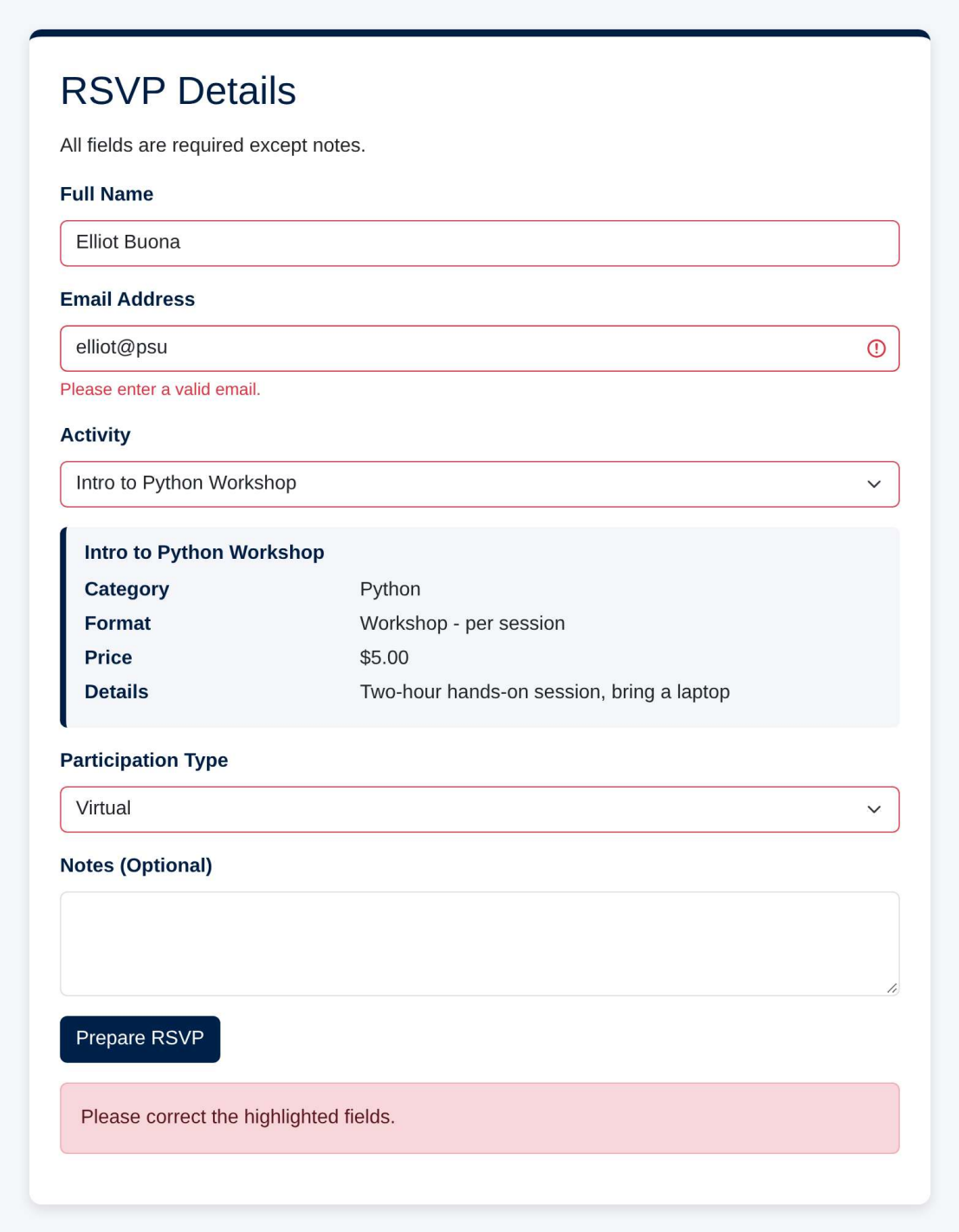
Upcoming Events page rendered in the browser with the navigation menu and RSVP form.

2. Form Validation

Name, email, activity, and participation type are required, and notes are limited to 500 characters. When the form is submitted, JavaScript checks every field. Invalid fields are marked in red with their own message, an alert asks the user to fix them, and no JSON is created until everything passes.



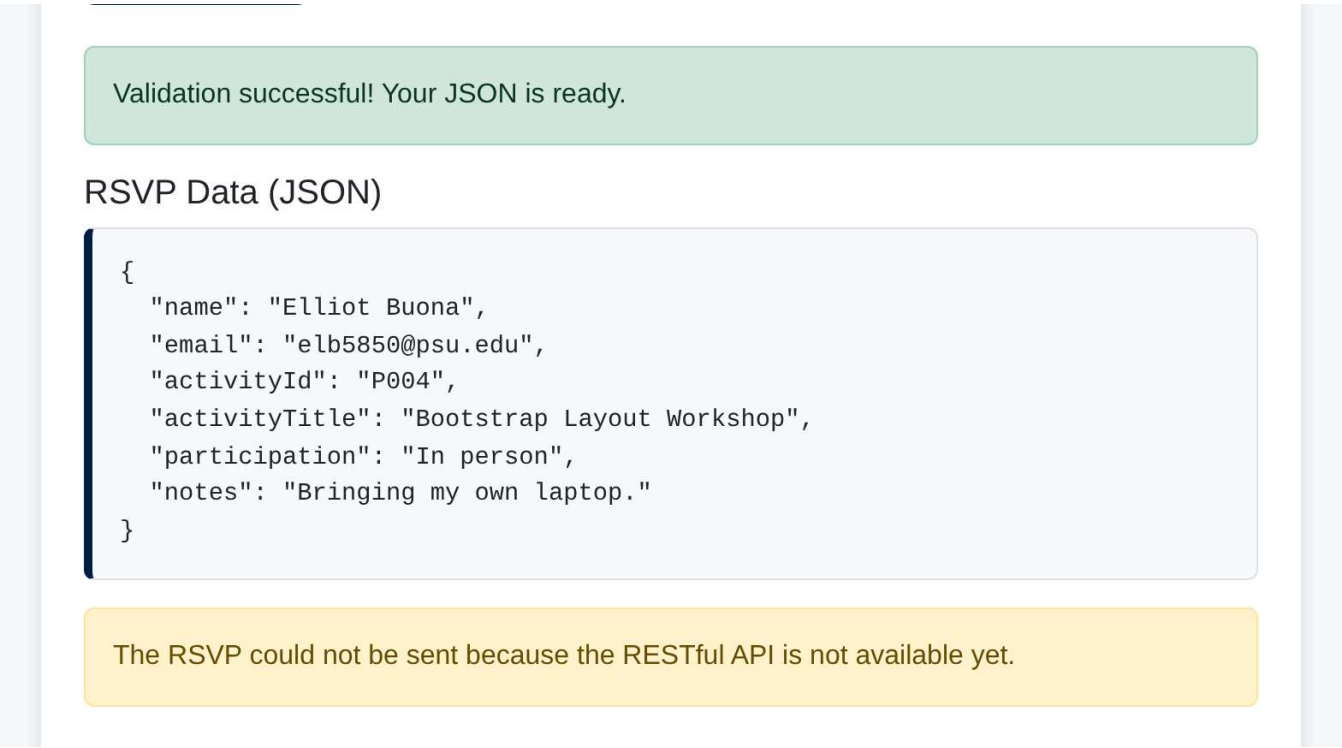
Error state: submitting an empty form marks all four required fields in red with a message for each.



Error state: "elliot@psu" is rejected as an invalid email while the other valid fields are accepted.

3. JSON RSVP Data

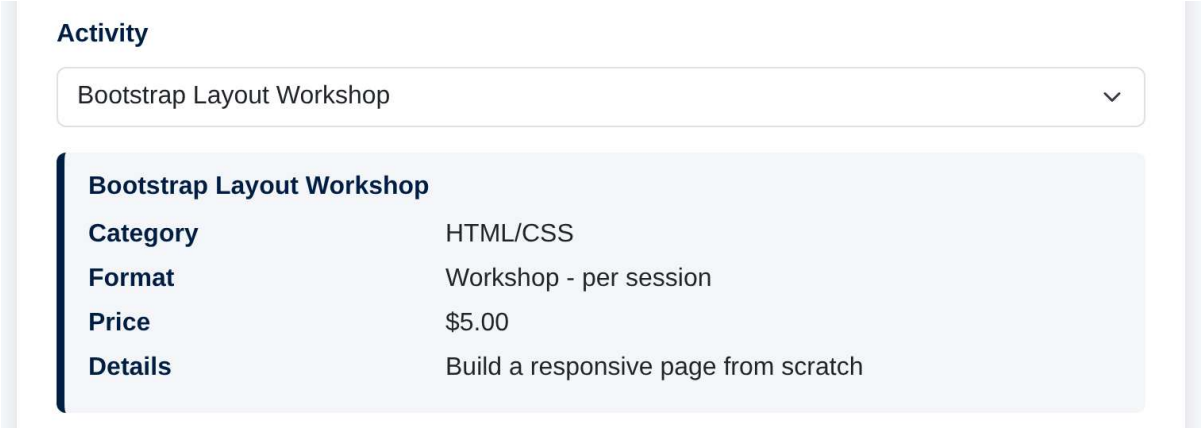
When validation passes, the user's selections are stored in a JSON object with the fields **name**, **email**, **activityId**, **activityTitle**, **participation**, and **notes**. The page shows the JSON so the user can review it before it is sent to the API.



RSVP Data (JSON) panel showing the RSVP object prepared for the API.

4. Activity Data with jQuery

The jQuery loads the club's existing product data from products.json with $.getJSON and keeps only the workshops, which become the choices in the Activity list. When a member picks a workshop, an activity details box shows that workshop's category, format, price, and description from the same data, so they can confirm what they are signing up for.



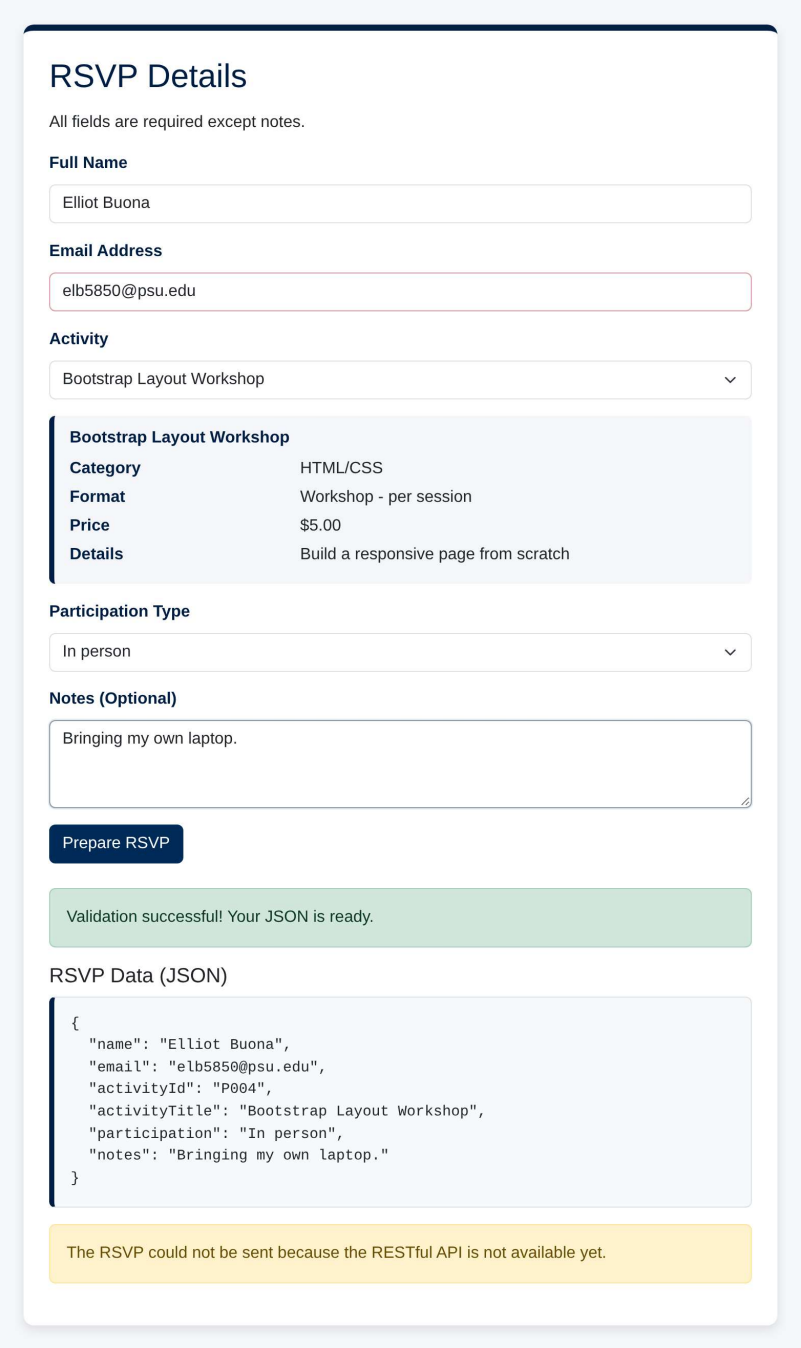
Activity details preview for the selected workshop, filled from products.json.

```json
[
  {
    "id": "P001",
    "title": "Python Crash Course Book",
    "category": "Python",
    "format": "Book - each",
    "price": 29.99,
    "notes": "Beginner book, used in the club's intro sessions"
  },
  {
    "id": "P002",
    "title": "Intro to Python Workshop",
    "category": "Python",
    "format": "Workshop - per session",
    "price": 5,
    "notes": "Two-hour hands-on session, bring a laptop"
  },
  {
    "id": "P003",
    "title": "HTML and CSS Starter Guide",
    "category": "HTML/CSS",
    "format": "Printed guide - each",
    "price": 12.5,
    "notes": "Covers page structure and styling"
  },
  {
    "id": "P004",
    "title": "Bootstrap Layout Workshop",
    "category": "HTML/CSS",
    "format": "Workshop - per session",
    "price": 5,
    "notes": "Build a responsive page from scratch"
  },
```

products.json: the existing product data. Items with a Workshop format are offered as RSVP activities.

5. AJAX

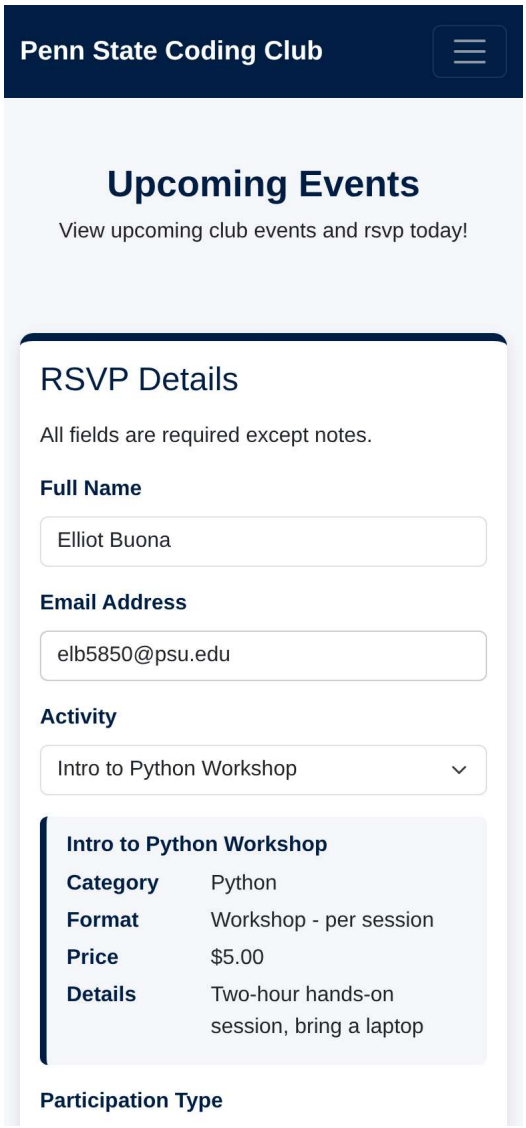
Submitting a valid RSVP sends the JSON to the RESTful API endpoint api/rsvp with a jQuery AJAX POST. The button is disabled while the request is sending. The API will be built in a later assignment, so the page shows a friendly message when the request cannot be completed.



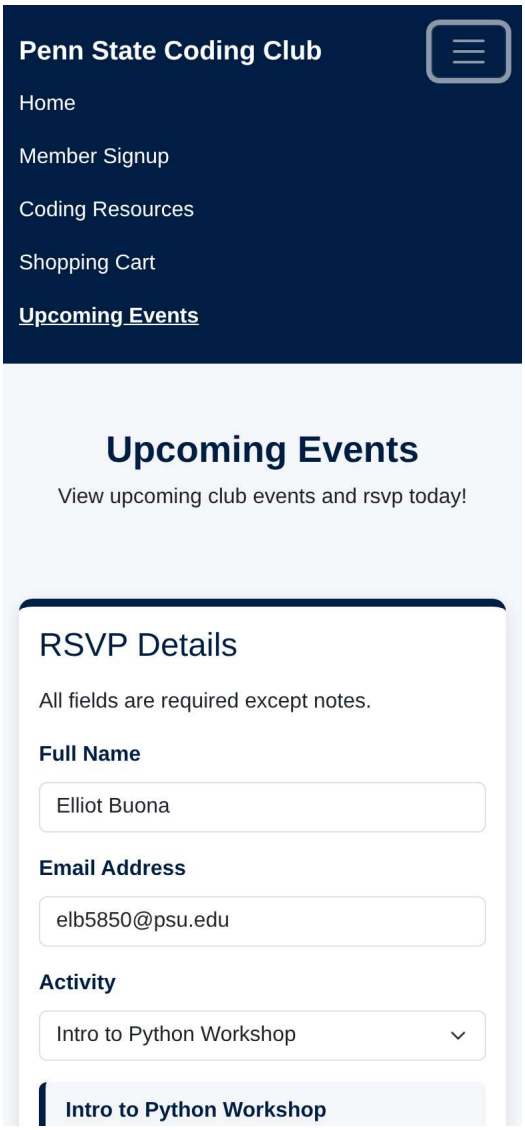
Valid RSVP: validation success, the generated JSON, and the AJAX result while the API is not yet available.

6. Responsive Design

Bootstrap's grid and the RSVP CSS keep the page usable on phones: the navbar collapses into a menu, the card uses less padding, and the button stretches to full width.



Phone width: stacked form and full-width button.



Phone width: navbar collapsed into a menu.

7. Code

HTML Page and Navigation - Adam Nisenson

```
    <!--Nav Bar -->
    <header>
        <nav class="navbar navbar-expand-lg bg-body-tertiary" data-bs-theme="dark">
            <div class="container">
                <a class="navbar-brand" href="index.html">Penn State Coding Club</a>

                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarMenu"
                    aria-controls="navbarMenu" aria-expanded="false" aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>

                <div class="collapse navbar-collapse" id="navbarMenu">
                    <ul class="navbar-nav ms-auto">
                        <li class="nav-item">
                            <a class="nav-link" aria-current="page" href="index.html">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" href="signup.html">Member Signup</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" href="resources.html">Coding Resources</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link" href="cart.html">Shopping Cart</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link active" href="rsvp.html">Upcoming Events</a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>
    </header>



    <main>
        <div class="container">

            <!-- Page heading -->
            <div class="row justify-content-center py-5">
                <div class="col-12 col-md-8 text-center">
                    <h1>Upcoming Events</h1>
                    <p>View upcoming club events and rsvp today!</p>
                </div>
            </div>

            <div id="rsvpApp">Loading RSVP form...</div>

        </div>
```

rsvp.html: navbar with Upcoming Events marked active, page heading, and the #rsvpApp container where React renders the form.

```
                <!-- Meeting Information -->
                <div class="col-md-6">
                    <div class="card h-100">
                        <div class="card-body">
                            <h2 class="card-title">Meeting Information</h2>
                            <p class="card-text"><strong>Day:</strong> To be announced</p>
                            <p class="card-text"><strong>Time:</strong> To be announced</p>
                            <p class="card-text"><strong>Location:</strong> To be announced</p>
                            <a href="signup.html" class="btn btn-primary">Join the Club</a>
                            <a href="rsvp.html" class="btn btn-primary">Upcoming Events</a>
                        </div>
                    </div>
                </div>
            </div>
```

index.html: Meeting Information card with the Upcoming Events button.

React Form and Validation - Zizhou Xiang

```
// Your name: React RSVP form, validation, and JSON generation.
const h = React.createElement;

// Madina: RESTful service endpoint (same style as cart.js)
const API_URL = "api/rsvp";

function RSVPApp() {
    // Madina: load activities from the existing products.json using jQuery
    const [activities, setActivities] = React.useState([]);
    const [loadError, setLoadError] = React.useState("");

    React.useEffect(function () {
        $.getJSON("products.json")
            .done(function (products) {
                // Only workshops can be RSVPed to (P002 and P004 right now)
                const workshops = products.filter(function (item) {
                    return item.format.startsWith("Workshop");
                });
                setActivities(workshops);
            })
            .fail(function () {
                setLoadError("Could not load activities. Please refresh the page.");
            });
    }, []);

    const [form, setForm] = React.useState({
        name: "",
        email: "",
        activity: "",
        participation: "",
        notes: ""
    });

    const [errors, setErrors] = React.useState({});
    const [rsvpData, setRsvpData] = React.useState(null);
    const [sending, setSending] = React.useState(false);
    const [sendStatus, setSendStatus] = React.useState(null);
```

rsvp.js: RSVPApp component with React state for the activities, form fields, errors, JSON data, and send status.

```
                            loadError
                        ),

                    h(
                        "form",
                        { onSubmit: handleSubmit, noValidate: true },

                        field("name", "Full Name"),
                        field("email", "Email Address"),

                        field(
                            "activity",
                            "Activity",
                            activities.map(function (item) {
                                return {
                                    value: item.id,
                                    label: item.title
                                };
                            })
                        ),

                        activityDetails(),

                        field("participation", "Participation Type", [
                            { value: "In person", label: "In person" },
                            { value: "Virtual", label: "Virtual" }
                        ]),

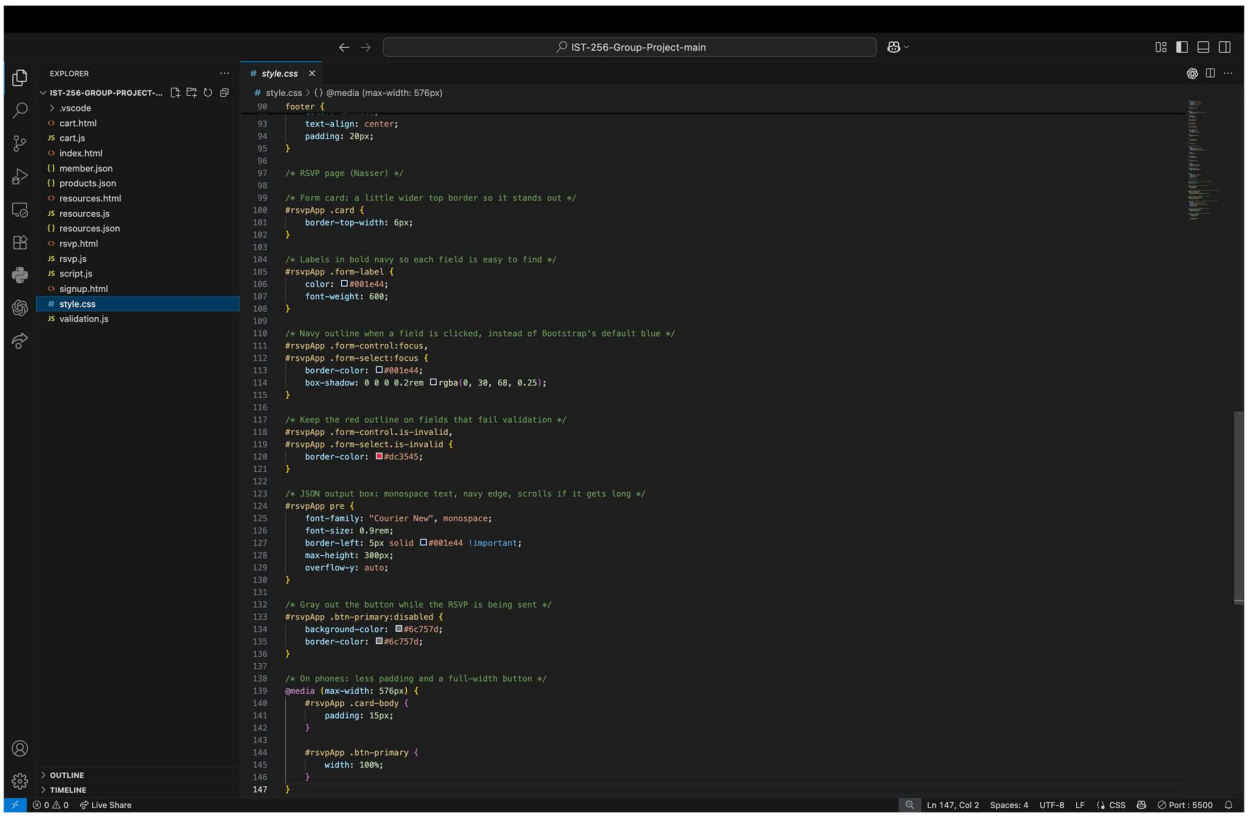
                        field("notes", "Notes (Optional)"),
```

rsvp.js: the form rendered with React: name, email, activity, participation type, and optional notes.

```
77      // Validate inputs and generate a JSON object.
78      function handleSubmit(event) {
79          event.preventDefault();
80
81          const found = {};
82
83          if (form.name.trim().length < 2) {
84              found.name = "Please enter your full name.";
85          }
86
87          if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email.trim())) {
88              found.email = "Please enter a valid email.";
89          }
90
91          const activity = activities.find(function (item) {
92              return item.id === form.activity;
93          });
94
95          if (!activity) {
96              found.activity = "Please select an activity.";
97          }
98
99          if (!["In person", "Virtual"].includes(form.participation)) {
100             found.participation = "Please select a participation type.";
101         }
102
103         if (form.notes.length > 500) {
104             found.notes = "Please use 500 characters or fewer.";
105         }
106
107         setErrors(found);
108
109         if (Object.keys(found).length > 0) {
110             setRsvpData(null);
111             return;
112         }
113
114         const payload = {
115             name: form.name.trim(),
116             email: form.email.trim(),
117             activityId: activity.id,
118             activityTitle: activity.title,
119             participation: form.participation,
120             notes: form.notes.trim()
121         };
122
123         setRsvpData(payload);
124         sendRsvp(payload);
125     }
```

rsvp.js: handleSubmit validates every field and builds the RSVP JSON object.

CSS and Page Fixes - Nasser Almalki



style.css: RSVP section scoped to #rsvpApp: labels, focus and error outlines, JSON box, sending state, and phone media query.

```html
<!doctype html>
<html lang="en" data-bs-theme="light">

<head>
    <title>Upcoming Events</title>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />

    <!-- Bootstrap CSS v5.3.8 -->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
        integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB" crossorigin="anonymous" />

    <link rel="stylesheet" href="style.css">


</head>
```

rsvp.html: page head with the corrected "Upcoming Events" title and Bootstrap CSS.

jQuery and AJAX - Madina Aliyeva

```
// Madina: RESTful service endpoint (same style as cart.js)
const API_URL = "api/rsvp";

function RSVPApp() {
    // Madina: load activities from the existing products.json using jQuery
    const [activities, setActivities] = React.useState([]);
    const [loadError, setLoadError] = React.useState("");

    React.useEffect(function () {
        $.getJSON("products.json")
            .done(function (products) {
                // Only workshops can be RSVPed to (P002 and P004 right now)
                const workshops = products.filter(function (item) {
                    return item.format.startsWith("Workshop");
                });
                setActivities(workshops);
            })
            .fail(function () {
                setLoadError("Could not load activities. Please refresh the page.");
            });
    }, []);
```

rsvp.js: $.getJSON loads products.json and keeps the workshops as RSVP activities.

```
    // Madina: send the JSON document to the RESTful API with AJAX
    function sendRsvp(payload) {
        setSending(true);
        setSendStatus(null);

        $.ajax({
            url: API_URL,
            method: "POST",
            contentType: "application/json",
            dataType: "json",
            data: JSON.stringify(payload)
        })
            .done(function () {
                setSendStatus({ type: "success", text: "RSVP submitted successfully." });
            })
            .fail(function () {
                setSendStatus({
                    type: "warning",
                    text: "The RSVP could not be sent because the RESTful API is not available yet."
                });
            })
            .always(function () {
                setSending(false);
            });
    }
```

rsvp.js: sendRsvp sends the RSVP JSON to the REST API with an AJAX POST.

Activity Details Preview - Elliot Buona

```
    function activityDetails() {
        const selected = activities.find(function (item) {
            return item.id === form.activity;
        });

        if (!selected) {
            return null;
        }

        function detail(label, value) {
            return [
                h("dt", { key: label + "Label", className: "col-4" }, label),
                h("dd", { key: label + "Value", className: "col-8" }, value)
            ];
        }

        return h(
            "div",
            {
                id: "activityDetails",
                className: "activity-details mb-3",
                "aria-live": "polite"
            },
            h("h3", { className: "h6" }, selected.title),
            h(
                "dl",
                { className: "row mb-0" },
                ...detail("Category", selected.category),
                ...detail("Format", selected.format),
                ...detail("Price", "$" + Number(selected.price).toFixed(2)),
                ...detail("Details", selected.notes)
            )
        );
    }
```

rsvp.js: activityDetails shows the selected workshop's details from products.json.

```
#rsvpApp .activity-details {
    background-color: #f4f6f9;
    border-left: 5px solid #001e44;
    border-radius: 6px;
    padding: 12px 15px;
}

#rsvpApp .activity-details h3 {
    color: #001e44;
    font-weight: bold;
}

#rsvpApp .activity-details dt {
    color: #001e44;
}

#rsvpApp .activity-details dd {
    margin-bottom: 4px;
}
```

style.css: styling for the activity details box.

All code is available at GitHub:
https://github.com/anisenson/IST-256-Group-Project.git